

Fase 1 — Auditoría y plan de evolución del panel interno

Documento maestro. Estado del sistema **verificado sobre el código real** (no suposiciones) y hoja de ruta para convertir el panel interno en la base de un **ERP propio del grupo Familia del Amor**. Alcance: **solo el panel interno** y su backend. No se toca la web pública, ni Ágora/Skello/Haddock (aún). No se reescribe nada; el monolito actual se **congela y se clasifica**.

1. Resumen ejecutivo

El panel interno funciona y da servicio diario, pero fue creciendo como un **monolito** (`server.js` , ~3560 líneas) con la lógica mezclada en las rutas y **autorización basada solo en el rol**. No existe aislamiento por establecimiento: hoy un `encargado` puede ver datos de todos los locales. La seguridad tiene deuda urgente (secreto JWT con fallback en el repo, sin rate-limit/helmet, un endpoint público de subida sin límites). No hay auditoría de acciones ni versionado de configuración.

La Fase 1 **no construye módulos nuevos**: audita, documenta, protege los flujos críticos (WhatsApp/Sara/reservas) con pruebas de regresión, y **diseña la arquitectura objetivo** (modular, basada en permisos, con feature flags, auditoría profesional y clean architecture) para no rehacerla dentro de un año. La implementación funcional se hará **después del gate**, en pasos pequeños y reversibles.

2. Arquitectura actual (verificada)

Capa	Tecnología	Notas
Backend	Node.js + Express (monolito <code>server.js</code>)	~85 endpoints, todas las rutas en un archivo. Sin capas (lógica en las rutas).
Chatbot	<code>whatsapp.js</code> (Baileys + Anthropic Claude)	Sin rutas HTTP; se conecta a <code>server.js</code> por callbacks (<code>setOnReserva</code> , etc.).
Facturación	<code>facturas.js</code> (Gmail + Drive + Sheets + Claude)	Motor de ingesta; sin rutas propias.
Frontend	Vanilla HTML/CSS/JS en <code>public/</code>	Una página por rol; Dirección embebe las demás por <code>iframe</code> . Sin framework.
BD	SQLite (<code>database.sqlite</code> , 28 tablas)	Sin ORM; SQL parametrizado a mano. "local" es texto libre repetido en 13 tablas, sin FK.
Auth	JWT (HS256, 8 h) en header <code>Authorization: Bearer</code>	Payload: <code>id, username, rol, nombre, local</code> .
Despliegue	Replit Reserved VM, arranque <code>node server.js</code>	Persistencia de BD por Replit-KV; sesión de WhatsApp se pierde en cada redeploy.

Ficheros clave: `server.js` , `whatsapp.js` , `facturas.js` , `seed-workers.js` , `public/*.js|html` , `database.sqlite` , `.replit` , `package.json` .

3. Hallazgos principales

3.1 Autorización (crítico para el objetivo)

- `requireAuth(roles)` (`server.js:1515-1533`) valida el JWT y comprueba **solo** `payload.rol` . El `local` del usuario viaja en el token pero **no se lee en ningún sitio** (0 referencias a `req.user.local`).
- **No hay aislamiento por establecimiento**. Endpoints como `GET /api/reservas` (2072), `GET /api/facturas` (1170), `GET /api/maintenance` (2388), `GET /api/reservas/export.csv` (2102) devuelven datos de **todos** los locales, o filtran por un `?local=` que envía el propio frontend sin validar acceso.
- No existe tabla de **establecimientos** ni de **asignación usuario→locales**.

3.2 Cuentas e identidad

- Cuentas **compartidas por rol** (un solo `encargado` , un solo `trabajador` genérico) con contraseña única `tapeta2024` (bcrypt coste 10). Sin trazabilidad por persona.
- Los **43 trabajadores nominales** sí tienen `local` asignado y coherente con los strings de las demás tablas → buena semilla para el backfill.

3.3 Seguridad urgente

- `JWT_SECRET` con fallback `"tapeta-secret-dev"` presente en el repo (`server.js:315`) → riesgo de **forja de tokens**.
- **Sin** rate-limit, helmet ni CORS (`server.js:317-319`).
- `POST /api/hr/applications` (2196) es **público** y usa multer **sin límite de tamaño ni filtro de tipo**, guardando en `public/uploads` (servido estáticamente).
- Los endpoints devuelven `e.message` interno al cliente; no hay manejador global de errores ni captura de `uncaughtException/unhandledRejection` .
- **Sin auditoría de acciones**. Refresh tokens de Google guardados/backupeados sin cifrar. PII (conversaciones, candidatos) en logs.

3.4 Persistencia

- Backup completo a Replit-KV **falla en silencio cuando la BD supera ~512 KB**. Existen backups paralelos de `config/leads/users`, pero otras tablas (`reservas`, `facturas`, `whatsapp_messages`...) pueden perderse en un redeploy. Sesión de WhatsApp no se respalda → re-linkado por QR tras cada redeploy.

(Detalle exhaustivo por módulo en `INVENTARIO_PANEL_INTERNO.md` ; flujos en `FLUJO_RESERVAS.md` y `FLUJO_WHATSAPP.md` ; riesgos y rollback en `RIESGOS_Y_ROLLBACK.md` .)

4. Objetivo de arquitectura (fundación ERP)

Principios que guían todo lo nuevo (detalle en `ARQUITECTURA_OBJETIVO_ERP.md`):

1. **Autorización por permisos, no por rol**. Usuario → Locales → Permisos → Rol (solo plantilla inicial). Nada de `if (rol === ...)` .
2. **Sistema por módulos** con acciones `modulo.accion` (ver/crear/editar/eliminar/exportar/administrar).
3. **Feature flags** para activar módulos/integraciones sin desplegar.
4. **Auditoría profesional** (usuario, fecha, IP, navegador, acción, antes/después).
5. **Versionado de configuración** restaurable.
6. **Todo configurable desde Dirección** (de-hardcode progresivo).
7. **Dashboard ejecutivo** real.
8. **AI-ready**: la capa de servicios es la superficie de acciones reutilizable por una IA.

9. **Clean architecture** para lo nuevo: Feature → API → Servicio → Repositorio → BD.
10. **API-first** para no limitar el rediseño visual futuro.

5. Hoja de ruta (orden estricto)

Paso	Entrega	Estado
1	Auditoría (estos documentos)	Esta fase
2	Documentar flujos WhatsApp y reservas (Mermaid)	Esta fase
3	Pruebas de regresión (baseline + spec de permisos)	Esta fase
—	GATE: revisión y autorización	Parada obligatoria
4	Núcleo src/core + módulo piloto en capas (sin cambiar comportamiento)	Post-gate
5	Seguridad urgente (JWT, rate-limit, helmet, CORS, uploads, errores)	Post-gate
6	Modelo de datos (empresas, establecimientos, user_locations, permisos, flags, audit_log, config_versions)	Post-gate
7	Motor de permisos en backend (requirePermission/requireLocationAccess/requireFinancialAccess) — grandfather	Post-gate
8	Pantalla Dirección → Administración (todo configurable)	Post-gate
9	Menú y filtros por permisos + flags	Post-gate
10	Dashboard ejecutivo v1 (datos reales)	Post-gate
11	Auditoría activada + base de versionado + documentar integraciones futuras	Post-gate

No se avanza si: no compila, fallan las pruebas baseline, WhatsApp se desconecta, las reservas dejan de funcionar, no hay rollback, o hay migración destructiva.

6. Criterios de aceptación de la Fase 1 (parte ejecutable)

- ☐ Los ~16 documentos existen en docs/ y reflejan el **código real**.
- ☐ Existen pruebas de regresión; las **baseline de reservas y WhatsApp pasan** en verde.
- ☐ La bandera TAPETA_TEST_MODE está **aislada** y no afecta a producción.
- ☐ No se ha modificado ningún endpoint, tabla ni la web pública.
- ☐ Todo en una **rama separada**; sin cambios en producción hasta aprobar.
- ☐ El diseño posterior (pasos 4-11) queda documentado y **gated**.

Arquitectura objetivo (fundación ERP)

Cómo se construye **todo lo nuevo**. No reescribimos el monolito actual (`server.js`): lo **congelamos y clasificamos**, y cada funcionalidad nueva (o cada módulo que migremos por necesidad) sigue esta arquitectura. Objetivo: escalabilidad, mantenibilidad, seguridad, rendimiento, modularidad y facilidad de extensión, sin sobreingeniería.

1. Principio: clean architecture por capas

```
Feature (dominio)
→ API      (routes finas: validan entrada, llaman al servicio, formatean salida)
→ Servicio (lógica de negocio; NO conoce Express ni SQL)
→ Repositorio (acceso a datos: SQL/consultas; NO conoce reglas de negocio)
→ Base de datos
```

Reglas:

- **Nada de lógica de negocio en las rutas.** La ruta orquesta; el servicio decide.
- **Nada de SQL fuera de los repositorios.**
- El **servicio** es puro y testeable sin arrancar el servidor (clave para pruebas y para AI-ready).
- Sin `if (rol === "...")` en ningún sitio: la autorización se resuelve con el motor de permisos (`MODELO_MODULOS_Y_PERMISOS.md`).

2. Estructura de carpetas (objetivo)

```
src/
  core/
    db.js          # conexión + helpers (dbGet/dbAll/dbRun ya existentes,
extraídos)
    auth.js        # verificación de JWT, req.user
    permissions.js # motor de permisos efectivos + requirePermission()
    locations.js   # acceso por local: requireLocationAccess(), locales efectivos
    financial.js   # requireFinanciamientoAccess()
    audit.js       # registro de auditoría (audit_log)
    flags.js       # feature flags (lectura/caché)
    config.js      # parámetros del sistema (get/set + versionado)
    errors.js      # manejador global de errores (forma {ok:false,error})
    validate.js    # validación de payloads (esquemas)
  modules/
    <dominio>/
      <dominio>.routes.js # define endpoints, aplica middlewares de core
      <dominio>.service.js # lógica de negocio
      <dominio>.repository.js # SQL
      <dominio>.schema.js # validación de entrada/salida
      <dominio>.actions.js # (futuro) descripción de acciones para IA
```

El monolito `server.js` sigue funcionando; `src/` se monta como router(s) adicional(es). Migración **incremental**, módulo a módulo, empezando por uno de bajo riesgo (mantenimiento o comunicados) como plantilla de referencia.

3. Contrato de API (consistencia)

- Respuestas siempre `{ ok: true, data }` o `{ ok: false, error, code }`.
- Nombres y formas consistentes entre módulos (paginación, filtros, fechas ISO).
- Cada endpoint declara: **permiso requerido** (`modulo.accion`), si es **por local**, y si es **financiero**.
- Esta consistencia es la que permite el **rediseño visual futuro** (front nuevo consumiendo la misma API) — ver `VISION_DISENO_Y_FRONTEND.md`.

4. AI-ready (preparación, no implementación)

La **capa de servicios** es, por diseño, la superficie de acciones del ERP. Una IA futura no habla con rutas ni con SQL: invoca servicios.

- Cada módulo podrá exponer un `<dominio>.actions.js`: un **registro de acciones** (nombre, descripción, parámetros validados, permiso requerido) que mapea 1:1 a funciones del servicio.
- Ese registro sirve para: (a) un futuro *tool/function registry* para IA (como ya hace Sara con sus tools, pero generalizado), (b) documentación automática de la API.
- **Requisito de diseño hoy**: servicios sin efectos colaterales ocultos, entradas/salidas explícitas y validadas, y **toda acción sensible pasa por el motor de permisos y por auditoría**. Así una IA queda automáticamente acotada por permisos y auditada.

5. Parámetros y catálogos (no-hardcode)

Todo lo que hoy está hardcodeado y tiene sentido administrar se mueve a datos configurables (ver `VERSIONADO_CONFIGURACION.md` y `FEATURE_FLAGS.md`):

- `LOCALES` (`whatsapp.js`) → tabla `establecimientos`.
- JIDs de personas (Nerea/Silvia/Laura) y `group_jid` por local → `config/establecimientos`.
- Umbrales (p.ej. >8 personas, horarios de reserva) → parámetros del sistema. De-hardcode **progresivo y no disruptivo**: primero coexisten valor en código y en BD (BD gana si existe), luego se retira el literal.

6. Rendimiento y datos

- SQLite es suficiente para el volumen actual; el patrón repositorio deja la puerta abierta a otro motor si algún día hace falta, sin tocar servicios.
- Índices en columnas de filtrado frecuente (`local / establecimiento_id`, fechas, teléfono) al introducir el modelo de datos.
- Cachés en memoria para feature flags y permisos efectivos (invalidación al cambiar).

7. Qué NO hacer (recordatorio)

- No migrar todo el monolito de golpe.
- No cambiar framework/BD/auth por gusto.
- No introducir capas o abstracciones que no resuelvan un problema real (sin sobreingeniería).

Modelo de acceso por establecimiento

El panel debe ser **realmente multiestablecimiento**: que un usuario solo vea los datos de los locales que tiene asignados, comprobado **siempre en backend**. Sin asumir que "como son del mismo grupo, todos ven todo".

1. Jerarquía conceptual

```
Organización          Grupo Familia del Amor
└ Empresa jurídica    (p.ej. Del Amor Uriel SLU, y otras sociedades del grupo)
  └ Establecimiento (cada restaurante / unidad operativa)
    └ Usuario ↔ Establecimiento (asignación con rol-por-local, activo,
    fechas)
```

- **Organización**: una (el grupo).
- **Empresa jurídica**: para contabilidad/facturación (cada establecimiento pertenece a una).
- **Establecimiento**: unidad operativa; es la clave del aislamiento.
- **Usuario**: cuenta de panel. **Perfil de trabajador**: información laboral de una persona (RRHH) — entidad distinta del usuario.

2. Establecimientos canónicos (semilla desde `LOCALES`, `whatsapp.js`)

#	Establecimiento (string actual)	Empresa jurídica
1	La Tapeta - Blanes	<i>a confirmar</i>
2	Cooperativa - Blanes	<i>a confirmar</i>
3	La Tapeta - Lloret	<i>a confirmar</i>
4	La Tapeta - Girona	<i>a confirmar</i>
5	Can Mateu - Tordera	<i>a confirmar</i>
6	La Tapa Ibérica - Tordera	<i>a confirmar</i>
7	Botiga d'en Mateu - Tordera	<i>a confirmar</i>

La relación establecimiento→empresa se tomará de `facturas_locales` (empresa/CIF) en **producción** (la copia local está vacía) y se confirmará con Dirección. El usuario mencionó **Del Amor Uriel SLU** como una de las sociedades. Nota: La Tapeta - Blanes y Cooperativa - Blanes comparten grupo de WhatsApp de reservas a propósito (no confundir con compartir empresa/aislamiento).

3. Modelo de datos (objetivo, migraciones aditivas)

```
empresas(id, nombre, cif, activo)
establecimientos(id, nombre, empresa_id, alias, activo,
  -- puente con el modelo actual:
  local_text UNIQUE)      -- el string actual ("La Tapeta - Blanes")
```

```
user_locations(id, usuario_id, establecimiento_id, rol_local NULL,
               activo, desde, hasta) -- un usuario, uno o varios locales
```

- `establecimientos.local_text` mapea 1:1 con los strings que hoy viven en 13 tablas → permite filtrar sin renombrar nada todavía.
- `user_locations` permite: **un usuario con un local, con varios, rol distinto por local** (opcional), **activación/desactivación** y **fechas de inicio/fin**.
- Se evita depender del único `users.local`. Ese campo se conserva (no se borra) durante la transición.

4. Backfill inicial

- Poblar `establecimientos` desde `LOCALES` (7 filas) y `empresas` desde `facturas_locales` /confirmación.
- Poblar `user_locations` para los **43 trabajadores** con su `users.local` actual (coincidencia por `local_text`).
- Cuentas de gestión genéricas (encargado, marketing...) → **sin asignación** por ahora (ver grandfather).

5. Estrategia *grandfather* (cero rotura — decisión del usuario)

- Usuario **sin** filas en `user_locations` → se comporta como hoy (**ve todo**), hasta que Dirección le asigne locales.
- En cuanto tenga ≥ 1 asignación, el backend **restringe** a esos locales.
- **Dirección** siempre tiene acceso **global** (no depende de asignaciones).
- La activación del filtrado va detrás de **feature flag** para poder desactivarla sin desplegar.

6. Dónde se aplica el aislamiento (evitar fugas indirectas)

El filtro por establecimiento debe alcanzar TODO lo que revele datos de un local: KPIs y contadores · buscadores · reservas · clientes · trabajadores · candidaturas · comunicados · incidencias · facturas · archivos · **exportaciones (CSV)** · grupos de WhatsApp · nombres de locales en selectores/autocompletados · filtros · estadísticas · logs visibles.

Regla dura: el `local / establecimiento_id` que llegue del frontend nunca concede acceso; el backend intersecta siempre con los locales efectivos del usuario y responde 403 (o filtra) si no procede.

7. Casos de prueba (ver `PLAN_PRUEBAS_REGRESION.md`)

Encargado Blanes solo Blanes · Encargado Lloret solo Lloret · Trabajador Blanes no ve Girona · Trabajador no ve facturación · Sin permiso → 403 · Manipular `establecimiento_id` no da acceso · Dirección ve todo · Usuario con dos locales ve solo esos dos.

Modelo de módulos y permisos

La pieza central del ERP: **la autorización se basa en permisos por módulo, no en el rol**. El rol solo sirve como **plantilla** de permisos por defecto al crear un usuario. Después, Dirección puede editar cualquier permiso de cualquier usuario sin tocar código.

1. Cadena de decisión

Usuario

- Locales asignados (¿sobre qué establecimientos?)
- Permisos efectivos (¿qué puede hacer en cada módulo?)
- Rol (solo plantilla inicial de permisos)

Prohibido `if (rol === "encargado")`. La pregunta siempre es: ¿este usuario tiene el permiso `módulo.accion` sobre este establecimiento?

2. Módulos (dominios)

Cada módulo es independiente y activable por feature flag. Catálogo inicial (ampliable sin cambiar arquitectura):

reservas, clientes, whatsapp, sara, rrhh, facturacion, contabilidad, marketing, mantenimiento, comunicados, kpis, web, usuarios, roles, permisos, integraciones, configuracion, ia.

3. Acciones por módulo

Set estándar de acciones (cada módulo usa las que le apliquen):

Acción	Significado
ver	Leer/listar
crear	Alta
editar	Modificar
eliminar	Borrar/cancelar
exportar	Descargar (CSV/...)
administrar	Configurar el módulo (ajustes, conexiones)

Permiso = `módulo.accion`. Ejemplos: `reservas.ver`, `reservas.crear`, `reservas.editar`, `reservas.eliminar`, `clientes.exportar`, `whatsapp.responder`, `facturacion.administrar`.

Permisos sensibles (financieros)

Los datos financieros (ventas, facturación, ticket medio, costes, márgenes, beneficio, facturas, pagos, IVA, gastos, costes laborales, escandallos, compras) requieren un permiso **explícito**:

`facturacion.verFinancieros` / `contabilidad.verFinancieros` / `kpis.verFinancieros`. El

acceso a un establecimiento NO concede acceso financiero. Por defecto: Dirección permitido; Contabilidad según permiso; resto denegado.

Acciones especiales de módulo (además de las estándar) cuando haga falta, p.ej.:

```
whatsapp.gestionarGrupos , whatsapp.gestionarConexion , facturacion.marcarPagado ,  
rrhh.verDatosPrivados , marketing.gestionarCampanas , marketing.gestionarWeb ,  
usuarios.gestionar , roles.gestionar , permisos.gestionar , integraciones.gestionar .
```

4. Permisos efectivos (cálculo)

```
permisos_efectivos(usuario) =  
  plantilla_de_rol(usuario.rol)           # defaults al crear  
  ⊕ concesiones_personalizadas(usuario)    # overrides que suman  
  ⊖ restricciones_explicitas(usuario)      # overrides que restan (ganan)  
  n acotado por locales asignados          # cada permiso se evalúa por establecimiento
```

- **La plantilla de rol es solo la semilla:** se copia a permisos del usuario en el alta. A partir de ahí, el usuario tiene sus propios permisos editables; cambiar la plantilla no altera retroactivamente a los usuarios ya creados (salvo acción explícita "restaurar valores por defecto").
- Las **restricciones explícitas** siempre ganan sobre las concesiones.
- El permiso se comprueba **junto con el local:** `requirePermission('reservas.editar', { establecimiento })` .

5. Modelo de datos (objetivo, migraciones aditivas)

```
role_templates(rol PK, permisos_json)           # plantilla por rol  
permisos(id, usuario_id, permiso, efecto ENUM(allow,deny), establecimiento_id NULL)  
# NULL = aplica a todos los  
locales del usuario
```

(Se puede empezar simple con `permisos_json` por usuario y evolucionar a filas si se requiere granularidad por local.)

6. Contrato de los middlewares (core)

- `requirePermission('modulo.accion')` — 403 si el usuario no tiene el permiso.
- `requireLocationAccess()` — resuelve los locales efectivos y **filtra**; ignora cualquier local / establecimiento_id del cliente no autorizado.
- `requireFinacialAccess()` — exige el permiso financiero explícito.
- Toda denegación y todo acceso sensible → **auditoría** (`AUDITORIA_PROFESIONAL.md`).
- **Grandfather:** un usuario sin permisos/locales asignados conserva el comportamiento actual (por compatibilidad) hasta que Dirección lo configure; Dirección siempre tiene acceso global.

7. Plantillas de rol iniciales (propuesta a validar)

Rol	Permisos por defecto (resumen)
direccion	Todo, incluido *.administrar y financieros. Acceso global.
encargado	reservas.*, whatsapp.ver/responder/gestionarGrupos, mantenimiento.*, comunicados.ver/crear, clientes.ver, kpis.ver (no financieros). Acotado a sus locales.

marketing	marketing.*, web.*, clientes.ver/editar/exportar, campanas.*, sara.administrar, reseñas. Global o por local.
rrhh	rrhh.* (incl. vacantes/candidaturas), comunicados.*. Global o por local.
contabilidad	facturacion.ver/marcarPagado/exportar, contabilidad.*, kpis.ver + financieros según permiso explícito . Por local/empresa.
trabajador	Configurable por Dirección: comunicados.ver, mantenimiento.crear, "ver mis incidencias", y futuros (horarios/vacaciones) solo como permisos preparados.

8. Preparación para trabajadores (solo permisos, sin funciones aún)

Dirección podrá decidir por trabajador: ver comunicados, crear incidencias, ver sus propias incidencias, y (cuando existan las integraciones) ver horarios/horas, solicitar vacaciones o cambios de turno, consultar documentos e información propia. **No se implementan esas funciones ahora**; solo se reservan los permisos para no rehacer el sistema.

Plan de migración

Todas las migraciones son **aditivas y reversibles** (`CREATE TABLE IF NOT EXISTS` , `ALTER TABLE ADD COLUMN`). Nunca `DROP` / `DELETE` / `RENAME` . Cada paso indica impacto, compatibilidad, rollback y recuperación de acceso. Se ejecutan **después del gate**, en commits separados.

0. Preparación (antes de cualquier migración)

- Rama separada (`fase-1-auditoria-erp`), sin tocar `main` hasta aprobar.
- Forzar backup KV + **copia local de `database.sqlite`** .
- Verificar que las pruebas baseline pasan.

1. Seguridad — `JWT_SECRET` obligatorio (paso 5)

- **Impacto:** si falta la env en prod, hoy usa fallback inseguro; queremos exigirla.
- **Compatibilidad:** ventana transitoria que acepta el secreto actual mientras se define el nuevo; avisar antes de forzar.
- **Rollback:** revertir a fallback temporal + reponer env.
- **Recuperación de acceso:** ver `RIESGOS_Y_ROLLBACK.md` §3 (resetear contraseña de un `direccion` por script).

2. Núcleo `src/core` + módulo piloto (paso 4)

- **Aditivo:** nuevo código en `src/` , montado como router adicional; el monolito sigue sirviendo el resto.
- **Sin cambio de comportamiento:** el módulo piloto (mantenimiento o comunicados) responde igual que hoy; pruebas antes/después idénticas.
- **Rollback:** revert del commit del módulo.

3. Modelo de datos (paso 6)

Tablas nuevas (conviven con las columnas `local` actuales, que **no se tocan**):

```
empresas(id, nombre, cif, activo)
establecimientos(id, nombre, empresa_id, alias, activo, local_text UNIQUE)
user_locations(id, usuario_id, establecimiento_id, rol_local NULL, activo, desde, hasta)
role_templates(rol PK, permisos_json)
permisos(id, usuario_id, permiso, efecto, establecimiento_id NULL) -- o
permisos_json por usuario
feature_flags(clave PK, activo, ambito, establecimiento_id NULL, descripcion, actualizado_en, actualizado_por)
audit_log(id, ts, usuario_id, username, ip, user_agent, accion, entidad_tipo, entidad_id, establecimiento_id, valor_anterior_json, valor_nuevo_json, resultado)
config_versions(id, ambito, clave, valor_json, version, creado_en, creado_por, comentario)
```

Índices: `establecimientos.local_text` , `user_locations(usuario_id)` , `audit_log(ts, usuario_id, accion)` .

Backfill (idempotente, verificable)

- `establecimientos` ← 7 filas de `LOCALES` (`whatsapp.js`).
- `empresas` ← `facturas_locales` (producción) + confirmación de Dirección.
- `user_locations` ← los **43 trabajadores** por su `users.local` (match por `local_text`).
- `role_templates` ← plantillas propuestas en `MODELO_MODULOS_Y_PERMISOS.md` .
- **Rollback:** `DROP` de tablas nuevas (aisladas); columnas `local` intactas; datos originales sin tocar.

4. Motor de permisos (paso 7)

- **Detrás de feature flag** `permisos_v2` . Con flag OFF, todo funciona como hoy (autorización por rol).
- **Grandfather:** usuario sin `user_locations` /permisos → acceso como hoy; Dirección global.
- Aplicación **módulo a módulo** (empezando por lectura de reservas/mantenimiento), con pruebas de "no filtra de más ni de menos".
- **Rollback:** desactivar el flag → vuelve al comportamiento por rol, sin desplegar.

5. Pantalla de Administración (paso 8)

- Solo nuevos endpoints/pantalla; no altera datos existentes.
- Cambios de permisos **auditados** y **versionados** → restaurables.

6. De-hardcode progresivo (transversal)

- `LOCALES` , JIDs (Nerea/Silvia/Laura), `group_jid` por local, umbrales → a `establecimientos` /config.
- **Coexistencia:** el código lee primero de BD; si no hay valor, usa el literal actual. Luego se retira el literal. Sin ventana de rotura.

7. Orden y gates

Cada paso: migración → backfill → pruebas verdes → commit → (si aplica) activar flag. **No se avanza** si falla una prueba baseline, se desconecta WhatsApp, o una migración no es reversible.

8. Qué NO se migra en esta fase

Nada funcional: esta fase entrega **solo** documentación y pruebas. Las migraciones anteriores son el **diseño** a ejecutar tras la aprobación del gate.

Riesgos y plan de rollback

Principio: **cada cambio debe ser reversible**. Ninguna migración destructiva. Los flujos críticos (WhatsApp/Sara/reservas) no se tocan sin pruebas verdes y rollback preparado.

1. Salvaguardas transversales

- **Rama de trabajo separada** (fase-1-auditoria-erp); main (compartido con Replit) no se toca hasta aprobar.
- **Backups Replit-KV** existentes: BD completa (latapeta_db_v3), config (latapeta_critical_config_v2), leads (latapeta_leads_v1), usuarios (latapeta_users_v1). Antes de cualquier migración, forzar backup y **exportar copia local** de database.sqlite .
- **Migraciones aditivas**: solo CREATE TABLE IF NOT EXISTS y ALTER TABLE ADD COLUMN . Nunca DROP / DELETE / RENAME / ALTER ... DROP . Cada migración con su reverso documentado.
- **Feature flags**: la lógica nueva (permisos, aislamiento) se activa detrás de flag → se puede **desactivar sin desplegar** si algo falla.
- **Gate de pruebas**: no se hace merge si fallan las baseline de reservas/WhatsApp.

2. Riesgos por cambio y su rollback

Cambio (paso)	Riesgo	Mitigación	Rollback
Test-mode flag (esta fase)	Alterar arranque en prod	Flag ausente en prod = comportamiento idéntico; aislado en su commit	Revertir el commit; el flag sin definir no hace nada
Núcleo src/core + piloto (4)	Regresión en el módulo migrado	Migrar módulo de bajo riesgo (mantenimiento/comunicados); pruebas antes/después iguales	Revert del commit del módulo; el resto del monolito intacto
JWT_SECRET obligatorio (5)	Dejar fuera a usuarios logueados / caída si falta la env	Definir la env antes de desplegar; ventana de compatibilidad que acepta el secreto viejo temporalmente; documentar recuperación de acceso	Revertir a fallback temporal + reponer env
rate-limit / helmet / CORS (5)	Bloqueos falsos, CORS roto	Límites holgados al inicio; CORS con allowlist propia; probar login y panel	Revertir middleware; son app.use aislados
multer límites/MIME + proteger /api/hr/applications (5)	Rechazar CVs legítimos; romper formulario público de empleo	Límites razonables (p.ej. 10 MB, PDF/imagen); mantener endpoint funcional con auth/anti-abuso, sin cambiar la web pública	Revertir config de multer
Manejador global de errores (5)	Cambiar formato de	Mantener forma {ok:false,error} para el cliente; dejar de filtrar	Revertir middleware

	respuestas de error	internos	
Modelo de datos nuevo (6)	Desalineación de strings de local; datos huérfanos	Tablas nuevas conviven con columnas local actuales (no se tocan); backfill idempotente y verificable	DROP de tablas nuevas (vacías/aisladas); columnas local intactas
Motor de permisos (7)	Dejar a alguien sin acceso (lockout)	Grandfather: sin asignaciones = acceso total como hoy; Dirección siempre con acceso global; activación por flag	Desactivar flag → vuelve al comportamiento por rol
Filtrado por local en queries (7)	Ocultar datos que antes se veían	Aplicar por módulo, con pruebas de "no filtra de más ni de menos"; primero en modo observación si hace falta	Revertir por módulo; flag global de bypass
Pantalla Administración (8)	Cambios de permisos erróneos	Versionado de config (restaurar estado anterior); auditoría de cada cambio	Restaurar snapshot de permisos
Menú/filtros por permisos (9)	Ocultar secciones necesarias	Grandfather + Dirección ve todo; validación con usuarios reales	Revertir front; backend no depende del menú
Dashboard ejecutivo (10)	Mostrar datos incorrectos	Solo datos existentes y fiables; nada de ventas/costes	Revertir vista; sin impacto en datos

3. Procedimiento de recuperación de acceso (si un cambio de auth deja fuera a Dirección)

1. Acceso al servidor (Replit) y a la BD.
2. Reponer/definir `JWT_SECRET` correcto en el entorno.
3. Si hace falta, resetear contraseña de un usuario `direccion` por `script ( bcrypt.hash )` directamente en `users`.
4. `restoreUsers()` /backup KV como red adicional.

4. Señales de parada (abortar y revertir)

- No compila / `node --check` falla.
- Falla cualquier prueba **baseline** de reservas o WhatsApp.
- WhatsApp se desconecta por un cambio nuestro.
- Una reserva no se crea/modifica/cancela o no notifica al grupo correcto.
- Aparece riesgo de pérdida de datos o una migración resulta no reversible.

Plan de pruebas de regresión

Red de seguridad **antes** de modificar nada funcional. Las pruebas **baseline** de reservas y WhatsApp deben pasar en verde; si fallan, se detiene el trabajo. Las de permisos/locales describen el **objetivo** (rojas hoy, verdes tras los pasos 6-7).

1. Herramientas

- **node:test** (runner integrado de Node, sin dependencia extra de framework).
- **supertest** para pruebas de endpoints HTTP (devDependency).
- **@playwright/test** para flujos críticos del panel (devDependency).
- Script `test` en `package.json` (hoy no existe ninguno).
- **BD temporal** por `test` (fichero SQLite en carpeta temporal), nunca `database.sqlite` real.

2. Aislamiento del test-mode (crítico)

TAPETA_TEST_MODE :

- Solo se define en el entorno de pruebas; en producción va **ausente**.
- Su único efecto (cuando está activa) es **no arrancar** WhatsApp/Baileys ni los cron, para poder testear contra BD temporal.
- **No cambia el comportamiento de producción, no toca WhatsApp/Sara/reservas, no introduce riesgo**. Se aísla en su propio commit.
- Preferencia: las pruebas de lógica atacan la **capa de servicios/repositorios** (sin efectos de arranque), reduciendo aún más la superficie.

3. Reservas — BASELINE (deben pasar hoy)

1. Crear reserva normal → fila en `reservas` con los campos correctos.
2. Reserva de **>8 personas** → `pendiente=true`, mensaje/aviso de pendiente.
3. Elegir **establecimiento** (cada `local` válido).
4. Elegir **zona** (terrazza/interior) solo en Blanesx2 y Lloret; resto `indiferente`.
5. **Fecha bloqueada** (`bloqueos_reservas`) → no se registra; motivo devuelto.
6. **Modificar** reserva (2→4 personas, hora, día, zona) → `UPDATE`, **misma fila** (no borra).
7. **Cancelar** → `DELETE`, fila eliminada.
8. **Conservar datos del cliente** (`wa_clientes` /lead) al reservar.
9. **Confirmación** al cliente construida correctamente.
10. **Notificar solo al grupo correcto** (`wa_links` del local) y **a ningún otro**.
11. **Follow-up** programado (+1 día 11:00) solo en confirmadas.
12. **Evitar duplicados** de notificación (cola `pending_whatsapp` si WA caído).

Nivel: unit sobre lógica pura de `whatsapp.js` (schemas de tools, `lineaZona`, formato de teléfono, diff de `modificar_reserva`) + integración replicando el handler contra BD temporal.

4. WhatsApp — BASELINE (socket mockeado)

Recibir mensaje · responder · mantener historial (`whatsapp_messages`) · identificar cliente · teléfono internacional (formato JID) · enviar PDF · enviar imagen · reconocer grupo · gestionar error temporal · no duplicar mensajes · conservar sesión cuando la infra lo permite. **Nunca** contra la sesión real.

5. Permisos / locales — SPEC OBJETIVO (rojas hoy)

- Encargado de Blanes ve **solo** Blanes.
- Encargado de Lloret ve **solo** Lloret.
- Trabajador de Blanes **no** ve Girona.
- Trabajador **no** ve facturación.
- Usuario sin permiso → **403**.
- Manipular `local / establecimiento_id` en query/body **no** da acceso a otro local.
- **Dirección** ve todos los establecimientos.
- Usuario con **dos locales** ve solo esos dos.
- Datos **financieros** denegados sin permiso explícito.

Estas pruebas se escriben ahora como especificación (se marcan como pendientes/ todo) y pasarán a verde cuando aterrice el motor de permisos (pasos 6-7).

6. Playwright (flujos de panel, cuando existan cambios de auth)

- Login por rol → aterriza en su panel.
- Encargado no ve secciones/datos de otro local.
- Dirección → Administración: asignar local a un usuario y comprobar el efecto.
- "Permiso denegado" se muestra correctamente.

7. Criterio de ejecución

- `npm test` → **verde en baseline** (reservas + WhatsApp). Las de permisos quedan como spec pendiente.
- Se ejecuta antes y después de cada cambio funcional futuro para probar **no regresión**.